

NASA HW3 REPORT

References:

- https://drive.google.com/file/d/1d818Zs_pBOLfk94XH22Z8_5Tk_FFvMbv/view
- <https://learnxinyminutes.com/bash/>
- https://en.wikipedia.org/wiki/Merkle_tree
- <https://medium.com/coinmonks/merkle-tree-a-simple-explanation-and-implementation-48903442bc08>
- <https://lbreede.github.io/www/posts/implementing-a-merkle-tree-in-python/>
- <https://ithelp.ithome.com.tw/articles/10222754>
- <https://blog.csdn.net/guoyihaoguoyihao/article/details/129342978>
- <https://goblinbook.union.build/light-clients/inclusion-proofs.html>



LTJ_TW 2025/9/27 下午 04:45

你應該是跟我一樣 後面的path都沒以 1/ 開頭，所以相對路徑是錯的



LTJ_TW 你應該是跟我一樣 後面的path都沒以 1/ 開頭，所以相對路徑是錯的



a97 2025/9/27 下午 05:38

超級感謝！

順便問一下測資的23號

是那個地方不符格式？

大概是我比較眼殘吧，

看了老半天沒看出來 (已編輯)

討論的同學：

- 陳鼎元、簡立佩、方婷

AI:

why orange in vscode

```
why under while [ $((kpow << 1)) -le "$t" ]  
this line everything changes to orange in  
vscode like something feels wrong but  
shellcheck doesnt show anything
```

已思考 11s

Good catch — nothing is actually *wrong* with Bash there, but VS Code's syntax highlighter is getting confused by the `<<` token inside the arithmetic expansion and is treating it like the start of a here-document. That makes the rest of the line (and sometimes the file) highlight in an odd colour even though `shellcheck` is fine with it.

Why it happens (short):

- `$(( ))` is arithmetic expansion and `<<` inside it is a *bit-shift* operator — valid in bash.
- Some editors / colourizers don't fully parse arithmetic expansion and still tokenize `<<` as the shell "here-doc" operator, which breaks highlighting.
- If the file is being parsed in a more-strict `sh` mode (or the extension uses a simpler parser), the colourizer is more likely to misinterpret `<<`.

What to do (practical fixes)

1. Use the arithmetic conditional `< ( )` instead of `[` with `$(( ))`.
It's clearer and avoids the confusing `<<` in a word context:

```
./merkle-dir.sh build dir1 --output
btw this line makes my parser go in a infinite
loop, can you find why
+ '[' -n '' ']'
+ '[' 1 -gt 0 ']'
+ case "$1" in
+ '[' -n '' ']'
+ '[' 1 -gt 0 ']'
+ case "$1" in
+ '[' -n '' ']'
+ '[' 1 -gt 0 ']'
+ case "$1" in
+ '[' -n '' ']'
+ '[' 1 -gt 0 ']'
+ case "$1" in
+ '[' -n '' ']'
partial output from set -x
```



Good catch — that exact command (`./merkle-dir.sh build dir1 --output`) reproduces the problem: the parser sees `--output` with no following value, the `case` arm does **nothing** (no shift, no error) and the `while` loop repeats forever with `$1` unchanged.

Why it loops

In your parsing loop this arm:

```
bash
```



複製程式碼

```
--output|--tree|--proof|--root)
```

can you write a script to compare your_output(directory with a lot of outputs in it) with output(same amount of output, 1 to 1) and check where is different?

Got it 👍 You want a script that compares **all files** in `your_output/` with the corresponding files in `output/` (1-to-1 match, same filenames), and tells you exactly which ones differ.

Here's a Bash script for that:

bash

複製程式碼

```
#!/usr/bin/env bash
```

```
your_dir="your_output"
```

```
expected_dir="output"
```

```
# iterate over files in your_output
```

```
for f in "$your_dir"/*; do
```

```
    fname=$(basename "$f")
```

```
    expected_file="$expected_dir/$fname"
```

```
    if [[ ! -f "$expected_file" ]]; then
```

```
        echo "MISSING expected file: $expected_file"
```

```
        continue
```

```
    fi
```

```
# compare with normalized newlines
```

double [[]] vs single [] diff

```
[ "${#args[@]}" -ne 1 ] || [ ! -f "${args[0]}" ] [[  
"${#args[@]}" -ne 1 || ! -f "${args[0]}" ]]
```

why
are these both ok?

Good catch 😊 — this comes down to the difference between

- the classic single-bracket test (`[ ... ]`, a synonym for the `test` command), and
- the double-bracket conditional expression (`[[ ... ]]`, a Bash/Ksh/Zsh keyword).

1. Single brackets `[ ... ]`

Each `[ ... ]` is a separate command.

So when you write:



will wildcard* match to other options

wont the wildcard also match to the second option?

Great question 👍 you're thinking ahead — whether the `*` branch (the wildcard in `case`) might accidentally swallow option values like `dir1.mktree` OR `file1.proof`.

Let's walk through it carefully.

Your parser loop

bash

複製程式碼

```
while [ "$#" -gt 0 ]; do
  case "$1" in
    --output|--tree|--proof|--root)
      if [ -n "$2" ] && [[ "$2" != --* ]]; then
        options+=("$1" "$2")
        shift 2
      fi
      ;;
    *)
      args+=("$1")
      shift
      ;;
  esac
```

`()` is not like in other languages

```
if [ "${#options[@]}" -ne 2 ] ||  
    [ "${options[0]}" != "--output" ] ||  
    ([ -e "${options[1]}" ] && [ ! -f  
"${options[1]}" ]); then  
    usage;  
    exit 1;  
fi
```

im pretty sure my logic is correct here, but it shows a syntax error, like ([-e "\${options[1]}"] && [! -f "\${options[1]}"]) this part is considered together instead of seperated how do i do that

Ah! I see exactly what's happening — the problem is **bash** doesn't allow parentheses `()` in `[ / test` expressions like you tried.

Let me explain carefully and show the fix.

1 Your code that gives a syntax error

bash

複製程式碼

```
if [ "${#options[@]}" -ne 2 ]
```



set -x output explanation, plus why hex is giving wrong output

this is the set -x for ./merkle-dir.sh build dir1
--output dir1.mktree. I noticed that the last
line seems off others are all correct. e.g
should be 61972
f8a5bf9925d70d34934dafced66756c5f9d7a
80e3b265e61e5526155f83 but mine is like
that.



Ah, I see exactly what's happening 👍

From your trace:

diff

複製程式碼

```
+++ xxd -r -p
+++ xxd -r -p
+++ printf %s 3606efb5...
++ parent=44de17c256699d1b...
```

The problem is the **order of operations** in the subshell that
computes `parent`. Right now you're running:

bash

複製程式碼

```
parent="$( ( printf '%s' "$left" | xxd -r -p ; printf '%s' "$right"
```

That's two separate pipelines:

- `printf "%s" "$left" | xxd -r -p` ↓
- `printf "%s" "$right" | xxd -r -p`

-h --help (but i so the problem later, ai is useless here ;;-;)

```
# Strict single-help check
if [ "$#" -eq 1 ] && { [ "$1" = "-h" ] || [ "$1" =
"--help" ]; }; then
    usage
    exit 0
fi

# Anything else starting with -h / --help is
invalid
if [[ "$1" == -* ]]; then
    usage
    exit 1
fi
```

this part is incorrect when i do ./merkle-dir.sh -h --help this should exit 1 but it exits 0. we have a lot of other commands to consider so be careful

Ah, I see exactly what's happening. Your current code only checks if **exactly one argument** is provided and that it is `-h` or `--help`. That's why `./merkle-dir.sh -h --help` matches the first `if` (because `$1` is `-h`)? Actually, let's carefully reason:

- `$# = 2` → fails the first `if` because it requires `-eq 1`. ✓
- Then it checks `if [[ "$1" == -* ]]` → `$1` is `-h` → matches → calls `usage` and exits 1. ✓

Hmm, based on your description it exits 0, which suggests maybe you have a subtle bug in brace grouping. ↓ `{ }` in the first `if`. In bash, the construct: